

API WebStorage

Damian Nicolalde-R.

danicolalde@puce.edu.ec

Temas:

- Sistemas de Almacenamiento
- Session Storage
 - Almacenando datos
 - Leyendo datos
 - Eliminando datos
- Local Storage
 - Evento storage

Sistemas de Almacenamiento

- La API Web Storage nos permite almacenar datos en el disco duro del usuario y acceder a los mismos cuando el usuario vuelve a visitar nuestro sitio web. El sistema de almacenamiento provisto por esta API se puede usar en dos situaciones particulares: cuando la información tiene que estar disponible solo durante una sesión y cuando se tiene que preservar hasta que lo determina el código o el usuario. La API se ha dividido en dos partes llamadas Session Storage y Local Storage.
- **Session Storage**—Este es un mecanismo de almacenamiento que mantiene los datos disponibles solo durante una sesión. A la información almacenada a través de este mecanismo se accede desde una ventana o pestaña, y se conserva hasta que se cierra la ventana.
- **Local Storage**—Este mecanismo trabaja de forma similar al sistema de almacenamiento de una aplicación de escritorio. Los datos se preservan de forma permanente y siempre están disponibles desde la aplicación que los ha creado.

Session Storage

- El sistema Session Storage es el más sencillo de todos. Este sistema almacena los datos solo para una sesión, lo cual significa que los datos se eliminarán cuando el usuario cierre la ventana o pestaña. Las aplicaciones pueden usarlo como soporte o para almacenar información que se puede solicitar más adelante en el proceso pero que se vuelve irrelevante si el usuario abandona el sitio web.

```

1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>API Web Storage</title>
6          <link rel="stylesheet" href="almacenamiento.css">
7          <script src="almacenamiento.js"></script>
8      </head>
9      <body>
10         <section id="cajaformulario">
11             <form name="formulario">
12                 <p><label>Clave: <input type="text" name="clave" id="clave"></label></p>
13                 <p><label>Valor: <textarea name="texto" id="texto"></textarea></label></p>
14                 <p><button type="button" id="grabar">Grabar</button></p>
15             </form>
16         </section>
17         <section id="cajadatos">
18             <p>Información no disponible</p>
19         </section>
20     </body>
21 </html>

```

Clave:

Valor:

Información no disponible

```

1  #cajaformulario {
2      float: left; padding: 20px;
3      border: 1px solid #999999;
4  }
5  #cajadatos {
6      float: left; width: 400px;
7      margin-left: 20px; padding: 20px;
8      border: 1px solid #999999;
9  }
10 #clave, #texto {
11     width: 200px;
12 }
13 #cajadatos > div {
14     padding: 5px;
15     border-bottom: 1px solid #999999;
16 }

```

Almacenando datos

- Los datos se almacenan como ítems, que se componen de un par nombre,valor. Estos ítems son como variables, cada uno con un nombre y un valor, que se pueden crear, modificar o eliminar. Los siguientes son los métodos con los que cuenta la API para crear y leer un ítem en el espacio de almacenamiento.
 - **setItem(nombre, valor)**—Este método crea y almacena un ítem con el nombre y el valor especificados por los atributos. Si ya existe un ítem con el mismo nombre, se actualizará con el nuevo valor, por lo que este método también se puede usar para modificar datos almacenados con anterioridad.
 - **getItem(nombre)**—Este método devuelve el valor del ítem con el nombre especificado por el atributo.
- El objeto **Window** incluye dos propiedades para facilitar acceso a los sistemas de almacenamiento: **sessionStorage** y **localStorage**. Para almacenar y leer ítems, tenemos que ejecutar los métodos correspondientes desde estas propiedades, como en el siguiente ejemplo.

```

1 function iniciar() {
2     var boton = document.getElementById("grabar");
3     boton.addEventListener("click", nuevoitem);
4 }
5 function nuevoitem() {
6     var clave = document.getElementById("clave").value;
7     var valor = document.getElementById("texto").value;
8     sessionStorage.setItem(clave, valor);
9     mostrar(clave);
10 }
11 function mostrar(clave) {
12     var cajadatos = document.getElementById("cajadatos");
13     var valor = sessionStorage.getItem(clave);
14     cajadatos.innerHTML = "<div>" + clave + " - " + valor + "</div>";
15 }
16 window.addEventListener("load", iniciar);

```

→ almacenamiento.js

Clave:

Valor:

Grabar

int - numero

- Además de estos métodos, la API también ofrece un atajo para crear y leer un ítem en el espacio de almacenamiento. Podemos usar el nombre del ítem como una propiedad de **sessionStorage** y acceder a su valor de esta manera. Como con cualquier otra propiedad, contamos con dos sintaxis: podemos encerrar la variable representando el nombre en corchetes (**sessionStorage[nombre] = valor**) o podemos usar notación de puntos (**sessionStorage.miitem = valor**).

```
1 function iniciar() {
2     var boton = document.getElementById("grabar");
3     boton.addEventListener("click", nuevoitem);
4 }
5 function nuevoitem() {
6     var clave = document.getElementById("clave").value;
7     var valor = document.getElementById("texto").value;
8     sessionStorage[clave] = valor;
9     mostrar(clave);
10 }
11 function mostrar(clave) {
12     var cajadatos = document.getElementById("cajadatos");
13     var valor = sessionStorage[clave];
14     cajadatos.innerHTML = "<div>" + clave + " - " + valor + "</div>";
15 }
16 window.addEventListener("load", iniciar);
```


Leyendo datos

- Los ítems se almacenan en un array, por lo que también podemos acceder a los valores con un índice o un bucle. La API ofrece esta propiedad y este método con dicho propósito.
 - **length**—Esta propiedad devuelve el número de ítems acumulados en el espacio de almacenamiento de la aplicación.
 - **key(índice)**—Este método devuelve el nombre del ítem en el índice especificado por el atributo.
- Los ejemplos anteriores solo leen el último ítem almacenado. Aprovechando el método **key()**, vamos a mejorar el código para listar todos los valores disponibles en el espacio de almacenamiento.

```

1  function iniciar() {
2      var boton = document.getElementById("grabar");
3      boton.addEventListener("click", nuevoitem); mostrar();
4  }
5
6  function nuevoitem() {
7      var clave = document.getElementById("clave").value;
8      var valor = document.getElementById("texto").value;
9      sessionStorage.setItem(clave, valor);
10     document.getElementById("clave").value = "";
11     document.getElementById("texto").value = "";
12     mostrar();
13 }
14
15 function mostrar() {
16     var cajadatos = document.getElementById("cajadatos");
17     cajadatos.innerHTML = "";
18     for (var f = 0; f < sessionStorage.length; f++) {
19         var clave = sessionStorage.key(f);
20         var valor = sessionStorage.getItem(clave);
21         cajadatos.innerHTML += "<div>" + clave + " - " + valor + "</div>";
22     }
23 }
24 window.addEventListener("load", iniciar);

```

Clave:

Valor:

Grabar

Esteban - Nicolalde
Damian - Nicolalde
string - nombres
int - numero

Eliminando datos

- Los ítems se pueden crear, leer y, por supuesto, eliminar. La API incluye dos métodos para eliminar ítems del espacio de almacenamiento.
 - **removeItem(nombre)**—Este método elimina un ítem. El atributo **nombre** especifica el nombre del ítem a eliminar.
 - **clear()**—Este método elimina todos los ítems del espacio de almacenamiento.

```

1  function iniciar() {
2      var boton = document.getElementById("grabar");
3      boton.addEventListener("click", nuevoitem);
4      mostrar();
5  }
6  function nuevoitem() {
7      var clave = document.getElementById("clave").value;
8      var valor = document.getElementById("texto").value;
9      sessionStorage.setItem(clave, valor);
10     document.getElementById("clave").value = "";
11     document.getElementById("texto").value = "";
12     mostrar();
13 }
14 function mostrar() {
15     var cajadatos = document.getElementById("cajadatos");
16     cajadatos.innerHTML = '<div><input type="button" onclick="removeTodo()" value="Eliminar Todos"></div>';
17     for (var f = 0; f < sessionStorage.length; f++) {
18         var clave = sessionStorage.key(f);
19         var valor = sessionStorage.getItem(clave);
20         cajadatos.innerHTML += "<div>" + clave + " - " + valor + "<br>";
21         cajadatos.innerHTML += '<input type="button" onclick="removeItem(\'' + clave + '\')" value="Remover"></div>';
22     }
23 }
24 function removeItem(clave) {
25     if (confirm("Está seguro?")) {
26         sessionStorage.removeItem(clave);
27         mostrar();
28     }
29 }
30 function removeTodo() {
31     if (confirm("Está seguro?")) {
32         sessionStorage.clear();
33         mostrar();
34     }
35 }
36 window.addEventListener("load", iniciar);

```

Clave:

Valor:

Benjamín - Nicolalde

Marcos - Nicolalde

Damián - Nicolalde

Jaime - Rivas

Local Storage

- Contar con un sistema fiable para almacenar datos durante una sesión puede ser útil en algunas circunstancias, pero cuando intentamos emular aplicaciones de escritorio en la Web, un sistema de almacenamiento temporario como este es generalmente insuficiente. Para mantener la información siempre disponible, la API Web Storage incluye el sistema Local Storage. Con Local Storage, podemos almacenar grandes cantidades de datos y dejar que el usuario decida si la información aún es útil y debemos conservarla o no.
- Este sistema usa la misma interfaz que Session Storage; por lo tanto, cada método y propiedad estudiados en este capítulo están disponibles también en Local Storage. Solo necesitamos sustituir la propiedad **sessionStorage** por la propiedad **localStorage** para preparar los códigos.

```

1  function iniciar() {
2      var boton = document.getElementById("grabar");
3      boton.addEventListener("click", nuevoitem);
4      mostrar();
5  }
6  function nuevoitem() {
7      var clave = document.getElementById("clave").value;
8      var valor = document.getElementById("texto").value;
9      localStorage.setItem(clave, valor);
10     mostrar();
11     document.getElementById('clave').value = "";
12     document.getElementById('texto').value = "";
13 }
14 function mostrar() {
15     var cajadatos = document.getElementById("cajadatos");
16     cajadatos.innerHTML = "";
17     for (var f = 0; f < localStorage.length; f++) { var clave = localStorage.key(f);
18         var valor = localStorage.getItem(clave);
19         cajadatos.innerHTML += "<div>" + clave + " - " + valor + "</div>";
20     }
21 }
22 window.addEventListener("load", iniciar);
--

```

La información estará disponible aun cuando ud cierre el navegador

Clave:

Valor:

b - 2

d - 4

e - 5

c - 3

a - 1

Evento storage

- Debido a que la información almacenada por Local Storage está disponible en todas las ventanas donde se carga la aplicación, debemos resolver dos problemas: cómo se comunican estas ventanas entre sí y cómo actualizamos la información en la ventana que no está activa. La API incluye el siguiente evento para solucionar ambos problemas.
 - **storage**—La ventana desencadena este evento cada vez que ocurre un cambio en el espacio de almacenamiento. Se puede usar para informar a cada ventana abierta con la misma aplicación que se ha realizado un cambio en el espacio de almacenamiento y que se debe hacer algo al respecto.
- Mantener la lista de ítems actualizada en nuestro ejemplo es fácil, solo tenemos que llamar a la función **mostrar()** cada vez que se desencadena el evento **storage**.

```

1  function iniciar() {
2      var boton = document.getElementById("grabar");
3      boton.addEventListener("click", nuevoitem);
4      window.addEventListener("storage", mostrar);
5      mostrar();
6  }
7  function nuevoitem() {
8      var clave = document.getElementById("clave").value;
9      var valor = document.getElementById("texto").value;
10     localStorage.setItem(clave, valor);
11     document.getElementById("clave").value = "";
12     document.getElementById("texto").value = "";
13     mostrar();
14 }
15 function mostrar() {
16     var cajadatos = document.getElementById("cajadatos");
17     cajadatos.innerHTML = "";
18     for (var f = 0; f < localStorage.length; f++) {
19         var clave = localStorage.key(f);
20         var valor = localStorage.getItem(clave);
21         cajadatos.innerHTML += "<div>" + clave + " - " + valor + "</div>";
22     }
23 }
24 window.addEventListener("load", iniciar);

```

En este ejemplo, la función **mostrar()** responde al evento **storage** y, por lo tanto, se ejecuta cada vez que se crea, actualiza o elimina un ítem. Ahora, si algo cambia en una ventana, se mostrará automáticamente en las otras ventanas que están ejecutando la misma aplicación.

Probar el código abriendo y actualizando los datos en varias ventanas

Clave:

Valor:

b - 20

d - 400

e - 5

c - 3

a - 3

Evento Storage

- El evento **storage** envía a la función un objeto de tipo **StorageEvent** que contiene las siguientes propiedades para proveer información acerca de la modificación producida en el espacio de almacenamiento.
 - **key**—Esta propiedad devuelve el nombre del ítem afectado.
 - **oldValue**—Esta propiedad devuelve el valor anterior del ítem afectado.
 - **newValue**—Esta propiedad devuelve el nuevo valor asignado al ítem.
 - **url**—Esta propiedad devuelve la URL del documento que ha producido la modificación. El espacio de almacenamiento se reserva para todo el dominio, por lo que pueden acceder a él diferentes documentos.
 - **storageArea**—Esta propiedad devuelve un objeto que contiene todos los pares nombre,valor disponibles en el espacio de almacenamiento después de la modificación.
- El siguiente ejemplo imprime en la consola los valores de estas propiedades para mostrar la información disponible. Probar en dos ventanas, modificas el valor en la primera y miras la consola de la segunda ventana.

```

1 function iniciar() {
2     var boton = document.getElementById("grabar");
3     boton.addEventListener("click", nuevoitem);
4     addEventListener("storage", modificado);
5     mostrar();
6 }
7 function nuevoitem() {
8     var clave = document.getElementById("clave").value;
9     var valor = document.getElementById("texto").value;
10    localStorage.setItem(clave, valor);
11    document.getElementById("clave").value = "";
12    document.getElementById("texto").value = "";
13    mostrar();
14 }
15 function modificado(evento) {
16     console.log(evento.key);
17     console.log(evento.oldValue);
18     console.log(evento.newValue);
19     console.log(evento.url);
20     console.log(evento.storageArea);
21     mostrar();
22 }
23
24 function mostrar() {
25     var cajadatos = document.getElementById("cajadatos");
26     cajadatos.innerHTML = "";
27     for (var f = 0; f < localStorage.length; f++) {
28         var clave = localStorage.key(f);
29         var valor = localStorage.getItem(clave);
30         cajadatos.innerHTML += "<div>" + clave + " - " + valor + "</div>";
31     }
32 }
33 window.addEventListener("load", iniciar);

```

